



Containers vs Virtual Machines

Introduction

In cloud computing and DevOps, two primary technologies are used for running applications in isolated environments:

- Virtual Machines (VMs)
- Containers

Both have their benefits, but **containers have revolutionized software deployment.**

- What are Virtual Machines (VMs)?
- What are Containers?
- How do they compare in performance, security, and resource usage?

This guide explains **the differences between containers and VMs** with real-world use cases.

Section 1: Learn

What is a Virtual Machine (VM)?

A **Virtual Machine (VM)** is a **software-based simulation** of a physical computer. It runs on a **hypervisor**, which allows multiple VMs to share the same physical hardware.

Component	Description
Hypervisor	Software that creates and manages VMs (e.g., VMware, VirtualBox)
Guest OS	Each VM has its own full operating system



Component	Description
Resource Usage	Each VM needs dedicated CPU, RAM, and storage
Isolation	Strong isolation, but resource-intensive

What is a Container?

A **Container** is a **lightweight, standalone software unit** that includes everything needed to run an application:

- Application code
- Libraries and dependencies
- System configurations

Unlike VMs, **containers share the host OS kernel**, making them **more efficient**.

Component	Description
Docker Engine	Runs and manages containers
Shared OS Kernel	Containers share the host OS instead of having separate OS instances
Resource Usage	Uses less CPU and memory
Speed	Starts instantly

Key Differences Between Containers and Virtual Machines

Feature	Virtual Machine (VM)	Container
Size	Large (GBs)	Small (MBs)
Boot Time	Slow (minutes)	Fast (seconds)



Feature	Virtual Machine (VM)	Container
OS Dependency	Requires full OS for each VM	Shares the host OS kernel
Resource Consumption	High	Low
Isolation	Strong (each VM is separate)	Moderate (shared OS)
Portability	Limited	High
Use Case	Running multiple OSes on one machine	Deploying applications efficiently

Why Are Containers Replacing VMs?

Challenge with VMs	Solution with Containers
Takes minutes to boot	Starts in seconds
Requires full OS installation	Shares host OS
Uses more CPU & RAM	Uses fewer resources
Difficult to move between environments	Runs the same anywhere

Interesting Fact

Google was **one of the first companies to use containers**. Today, **over 75% of enterprises use containers** for deploying applications.



Section 2: Practice

1. Running a Virtual Machine

Using VirtualBox to create a VM:

1. Install **Oracle VirtualBox**.
2. Download an **Ubuntu ISO** from the official website.
3. Open VirtualBox → Click **New** → Select **Ubuntu ISO** → Configure RAM & Disk.
4. Start the VM.

Issues with this approach:

- Takes time to **install and configure** the VM.
- Requires **high CPU and RAM usage**.

2. Running a Container

Using Docker to create a container:

```
docker run -d -p 8080:80 nginx
```

This runs an **Nginx web server** instantly in a container.

Why is this better?

- **No OS installation required**
- **Starts in seconds**
- **Uses fewer system resources**

3. Checking System Resource Usage

To see the difference in resource usage:



Check Virtual Machines (on Linux/Windows)

```
top
```

This shows high CPU and RAM usage for VMs.

Check Running Containers

```
docker ps
```

```
docker stats
```

This shows **how efficiently Docker containers run** compared to VMs.

Section 3: Know More

Frequently Asked Questions

1. **Can containers replace VMs completely?**
 - Not always. **VMs are still used for running multiple operating systems** on one machine.
2. **Are containers more secure than VMs?**
 - VMs provide **stronger isolation**, but **containers are secure when configured properly**.
3. **Can I run Docker inside a VM?**
 - Yes, many cloud providers run **Docker inside lightweight VMs** for better security.
4. **Why are containers faster than VMs?**
 - They **skip the OS boot process** and use **fewer system resources**.
5. **Which is better for cloud applications: VMs or Containers?**



- **Containers** are preferred for **modern cloud applications** due to their **portability and speed**.
-

Conclusion

Containers **offer a lightweight, fast, and scalable alternative to VMs**, making them ideal for **cloud computing and DevOps**.

Start by **running containers using Docker**, comparing their performance with **VMs**, and **deploying cloud-native applications efficiently!**